Rasterizer Math

software-rasterizer.c

Matti Fischbach
matti.fischbach@web.de

Contents

1 Vector Math

All positions and directions are `vec3f` — three floats. Defined in `math/vec.h`. The `vec_*` macros dispatch to the typed implementations via `_Generic`.

1.1 Addition and Subtraction

Component-wise. Used everywhere: translate a point, compute a direction.

$$\boldsymbol{a} + \boldsymbol{b} = \begin{pmatrix} a_x + b_x \\ a_y + b_y \\ a_z + b_z \end{pmatrix}$$

1.2 Dot Product

Scalar result. Measures how much two vectors point in the same direction.

$$\boldsymbol{a} \cdot \boldsymbol{b} = a_x b_x + a_y b_y + a_z b_z$$

If both vectors are unit-length: $\boldsymbol{a} \cdot \boldsymbol{b} = \cos\theta$ where θ is the angle between them. Used for backface culling: if $\boldsymbol{n} \cdot \boldsymbol{v} < 0$ the triangle faces away from the camera.

1.3 Cross Product

Returns a vector **perpendicular** to both inputs. Magnitude equals the area of the parallelogram they span.

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Used to compute face normals from two triangle edges.

1.4 Normalize

Scales a vector to unit length (magnitude 1).

$$\hat{\boldsymbol{v}} = \frac{\boldsymbol{v}}{|\boldsymbol{v}|}, \quad |\boldsymbol{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

Guard against division by zero: if $|\boldsymbol{v}| < 0.00001$ return $(0, 0, 0)$.

2 Coordinate Spaces

Every vertex passes through four coordinate spaces before it becomes a pixel. Each space exists for a reason. Transforming between them is the core of the pipeline.

Object Space
↓ scale, rotate, translate
World Space
↓ subtract cam pos, rotate by −pitch/−yaw
Camera Space
↓ perspective divide
NDC
↓ remap $[-1, 1]$ to pixels
Screen Space

Figure 1: The four coordinate spaces in the pipeline.

2.1 Object Space

Vertices as stored in the `.obj` file. The mesh is defined around its own origin (e.g a cube centered at $(0, 0, 0)$, a character with feet at $y = 0$, etc.)

```c
typedef struct world_s {
    ...,
    mesh.vertices[],
    ...,
}
```

2.2 World Space

Object space after the *instance transform* is applied. Every placed object (a `mesh_instance`) has its own position, rotation, and scale that move the mesh into the shared scene coordinate system.

The transform is applied in this order (intrinsic YXZ — each rotation is in the **already-rotated** local frame):

```c
// from transform_triangle_to_camera() in renderer/draw.c
v = rotate_z_snapped(v, inst->rotation.z * DEG_TO_RAD); // 1. roll
v = rotate_x_snapped(v, inst->rotation.x * DEG_TO_RAD); // 2. pitch
v = rotate_y_snapped(v, inst->rotation.y * DEG_TO_RAD); // 3. yaw
vec3f world_pos = vec_add(v, inst->pos);                // 4. translate
```

The world coordinate system is right-handed: $+x$ right, $+y$ up, $+z$ forward.

2.2.1 Degrees to radians (`DEG_TO_RAD`)

Instance rotations are stored in degrees — intuitive for editing. `sinf`/`cosf` require radians. Every angle is multiplied by `DEG_TO_RAD` before being passed to a rotation function:

```c
#define DEG_TO_RAD 0.017453292f  /* PI / 180 */
```

The conversion factor comes from the definition of a radian:

$$1^\circ = \frac{\pi}{180} \approx 0.017453 \text{ rad}$$

So $90^\circ \times \text{DEG\_TO\_RAD} = \frac{\pi}{2}$, $180^\circ \times \text{DEG\_TO\_RAD} = \pi$, etc.

2.2.2 Yaw (rotation around Y)

Yaw rotates a vector around the vertical (Y) axis — turning left/right.

$$\begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x\cos\theta + z\sin\theta \\ y \\ -x\sin\theta + z\cos\theta \end{pmatrix}$$

```c
// from rotate_y() in math/rotation.c
vec3f rotate_y(vec3f v, float yaw) {
  float cosine = cosf(yaw), sine = sinf(yaw);
  return (vec3f){
    v.x * cosine + v.z * sine,
    v.y,
    -v.x * sine + v.z * cosine,
  };
}
```

2.2.3 Pitch (rotation around X)

Pitch rotates around the horizontal (X) axis — looking up/down. Applied to the already-yawed vector.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi & \cos\varphi \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x \\ y\cos\varphi - z\sin\varphi \\ y\sin\varphi + z\cos\varphi \end{pmatrix}$$

```c
// from rotate_x() in math/rotation.c
vec3f rotate_x(vec3f v, float pitch) {
  float cosine = cosf(pitch), sine = sinf(pitch);
  return (vec3f){
    v.x,
    v.y * cosine - v.z * sine,
    v.y * sine + v.z * cosine,
  };
}
```

2.2.4 Roll (rotation around Z)

Roll rotates around the depth (Z) axis — tilting left/right. Applied first in the instance transform, before pitch and yaw.

$$\begin{pmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x\cos\psi - y\sin\psi \\ x\sin\psi + y\cos\psi \\ z \end{pmatrix}$$

z is unchanged — the vector just spins in the x-y plane.

```c
// from rotate_z() in math/rotation.c
vec3f rotate_z(vec3f v, float roll) {
  float cosine = cosf(roll), sine = sinf(roll);
  return (vec3f){ v.x * cosine - v.y * sine, v.x * sine + v.y * cosine, v.z };
}
```

The snapped variant (`rotate_z_snapped`) replaces `sinf`/`cosf` with `sincos_snap` for the same reason as the yaw/pitch snapping — instance roll values are typically 0°, 90°, 180°, 270° and must not drift.

2.2.5 Why roll → pitch → yaw?

This matches how a first-person camera works. Yaw turns your body; pitch then tilts your head relative to that turned body. Reversing the order would cause "gimbal" — if you pitched first, yawing would roll the horizon.

2.2.6 Snapped rotations

Instance objects are often rotated by exact multiples of 90°. A problem arises: `sinf` and `cosf` on floating-point hardware do not return exact results even for "clean" angles.

```
sinf(0.0f)       → 0.0          ✓  (lucky)
sinf(M_PI)       → -8.74e-8     ✗  (should be 0)
cosf(M_PI / 2)   →  6.12e-17    ✗  (should be 0)
```

When these near-zero values multiply a vertex coordinate, the result is a vertex that **should** sit exactly on an axis but is instead displaced by a tiny floating-point error. That causes visible seams and z-fighting on meshes aligned to the grid.

`sincos_snap` detects when an angle is within `SNAP_THRESHOLD` (0.01°) of a 90° multiple and substitutes exact lookup values instead of calling `sinf`/`cosf`:

```c
// math/rotation.h
#define SNAP_THRESHOLD 0.01f
static const float sine_lookup[4]   = { 0.0f,  1.0f,  0.0f, -1.0f };
static const float cosine_lookup[4] = { 1.0f,  0.0f, -1.0f,  0.0f };
```

The quadrant index q is derived from the angle in degrees:

$$q = \text{round}(\theta_{\text{deg}}/90) \bmod 4$$

So $0° \to q = 0$, $90° \to q = 1$, $180° \to q = 2$, $270° \to q = 3$. The lookup returns exactly $\{0, 1, -1\}$ — no float error.

`sine_lookup` and `cosine_lookup` are the table names — defined in `math/rotation.h` alongside `SNAP_THRESHOLD` so both threshold and tables are easy to tune in one place.

The `if (q < 0) q += 4` guard handles negative angles (e.g. camera rotating left, `-90°`). C's `%` operator keeps the sign: $-1 \bmod 4 = -1$, which is an out-of-bounds index. Adding 4 wraps it back into $[0, 3]$:

raw q	after wrap	angle
-1	3	270°
-2	2	180°
-3	1	90°

`rotate_x_snapped`, `rotate_y_snapped`, and `rotate_z_snapped` are identical to their non-snapped counterparts but call `sincos_snap` instead of `sinf`/`cosf` directly.

Only instance rotations use this. The camera uses plain `rotate_x`/`rotate_y` because camera angles change continuously and essentially never land on an exact cardinal.

2.3 Camera Space

The camera sits at the origin of camera space. The whole world is shifted and rotated so that the camera is at $(0, 0, 0)$ looking down $+z$.

(a) translate: subtract camera position from the world-space vertex.

$$\boldsymbol{p}_{\text{rel}} = \boldsymbol{p}_{\text{world}} - \boldsymbol{p}_{\text{cam}}$$

(b) rotate: undo the camera's own orientation by rotating by $-$pitch and $-$yaw. This is the inverse of how the camera is aimed.

```c
// from transform_triangle_to_camera() in renderer/draw.c
vec3f relative_pos = vec_sub(world_pos, cam->pos);
out[j] = rotate_x(rotate_y(relative_pos, -cam->yaw), -cam->pitch);
```

After this: if a vertex is directly in front of the camera, it has $x = 0, y = 0, z > 0$. A vertex to the left has $x < 0$. Above the camera: $y > 0$.

Camera space is right-handed with $+z$ pointing forward (into the scene).

2.3.1 Backface Culling

Once in camera space, triangles facing away from the camera can be skipped entirely — their back side is never visible on a closed mesh.

Step 1 — face normal: cross product of two edges from the same vertex gives a vector perpendicular to the triangle surface. For CCW winding it points toward the viewer.

$$\boldsymbol{e}_1 = \boldsymbol{V}_1 - \boldsymbol{V}_0, \quad \boldsymbol{e}_2 = \boldsymbol{V}_2 - \boldsymbol{V}_0, \quad \hat{\boldsymbol{n}} = \text{normalize}(\boldsymbol{e}_1 \times \boldsymbol{e}_2)$$

Step 2 — centroid: average of the three vertices. Used as the sample point for the view direction — more accurate than any single vertex.

$$\boldsymbol{c} = \frac{\boldsymbol{V}_0 + \boldsymbol{V}_1 + \boldsymbol{V}_2}{3}$$

Step 3 — view direction: camera is at origin in camera space, so the vector from the centroid to the camera is simply $-\boldsymbol{c}$:

$$\hat{\boldsymbol{v}} = \text{normalize}(-\boldsymbol{c})$$

Step 4 — dot product test:

$$\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{v}} < 0 \quad \rightarrow \quad \text{back face} \rightarrow \text{cull}$$

Negative dot product means the normal points away from the camera. If positive, the face is visible.

```c
// from is_backfacing() in renderer/geometry.c
vec3f view_dir = vec_normalize(vec3f_scale(centroid, -1.0f));
return vec_dot(normal, view_dir) < 0.0f;
```

2.4 NDC (Normalized Device Coordinates)

NDC maps the visible frustum to a unit cube $[-1, 1]^2$. This is where perspective projection happens — the core of the "things far away look smaller" effect.

The *perspective divide* divides x and y by z:

$$x_{\text{proj}} = \frac{x_{\text{cam}}}{z_{\text{cam}}} \cdot f, \quad y_{\text{proj}} = \frac{y_{\text{cam}}}{z_{\text{cam}}} \cdot f \cdot r$$

where f is `focal_length` (controls field of view) and r is `aspect_ratio` ($\frac{\text{screen\_width}}{\text{screen\_height}}$, keeps pixels square).

```c
// from project() in math/projection.c
float x_proj = (cam_pos.x / cam_pos.z) * world->cam->focal_length;
float y_proj = (cam_pos.y / cam_pos.z) * world->cam->focal_length
               * world->renderer->aspect_ratio;
```

Why does dividing by z create perspective? Because a vertex at $(1, 0, 2)$ (twice as far as one at $(1, 0, 1)$) projects to $x = \frac{1}{2}$ — half the screen position. Things that are further away shrink proportionally to their distance.

After the divide, $x_{\text{proj}} \in [-1, 1]$ and $y_{\text{proj}} \in [-1, 1]$ define what is inside the camera's view. Anything outside is clipped.

2.5 Screen Space

The final step maps NDC to integer pixel coordinates. The origin moves to the **top-left** corner, and $+y$ points **down** (screen convention).

$$x_{\text{screen}} = (x_{\text{proj}} + 1) \cdot 0.5 \cdot W$$

$$y_{\text{screen}} = (1 - y_{\text{proj}}) \cdot 0.5 \cdot H$$

where $W =$ `screen_width` and $H =$ `screen_height`.

The y axis is **flipped** $(1 - y_{\text{proj}})$ because NDC has $+y$ pointing up but screen pixels count down from the top.

```c
// from project() in math/projection.c
out->x = (x_proj + 1.0f) * 0.5f * world->renderer->screen_width;
out->y = (1.0f - y_proj) * 0.5f * world->renderer->screen_height;
out->z = cam_pos.z;  // keep original depth for depth test
```

Note: `out->z` is **not** projected — it keeps the original camera-space depth value. This is needed later for the depth buffer (closer z wins).

3 Near-Plane Clipping

Vertices behind the camera have $z <$ near in camera space. The perspective divide $\left(\frac{x}{z}\right)$ on a negative or near-zero z produces garbage screen coordinates and inverts geometry. Triangles that straddle the near plane must be clipped before projection.

3.1 Sutherland-Hodgman (1D)

The algorithm walks each edge of the input triangle and tests both endpoints against the near plane ($z \geq$ near). Four cases:

prev	curr	emit
inside	inside	curr
inside	outside	intersection
outside	inside	intersection + curr
outside	outside	nothing

The intersection point is found by linear interpolation along the edge:

$$t = \frac{\text{near} - z_{\text{prev}}}{z_{\text{curr}} - z_{\text{prev}}}, \quad \boldsymbol{p}_{\text{clip}} = \boldsymbol{p}_{\text{prev}} + t(\boldsymbol{p}_{\text{curr}} - \boldsymbol{p}_{\text{prev}})$$

```c
// from clip_triangle_near_plane() in renderer/geometry.c
float t = (near - prev.z) / (curr.z - prev.z);
out[out_count++] = (vec3f){
    prev.x + t * (curr.x - prev.x),
    prev.y + t * (curr.y - prev.y),
    near,
};
```

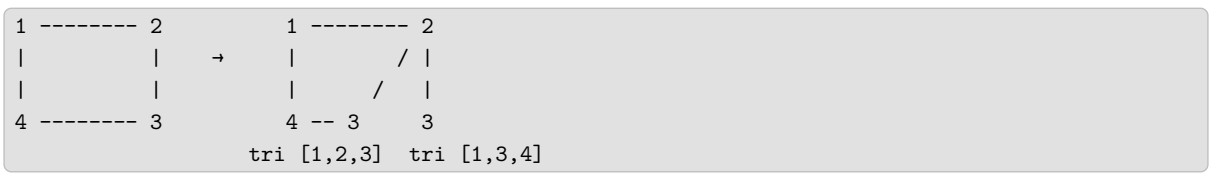
A triangle (3 vertices) can produce up to 4 output vertices after clipping — a quad. That quad is then fan-triangulated into 2 triangles before rasterization.

3.2 Fan Triangulation

Clipping can produce 4 output vertices (a quad). The rasterizer only handles triangles, so the quad is split into 2 triangles by anchoring at vertex 0 and walking the rest:

```
t=0 → triangle [0, 1, 2]
t=1 → triangle [0, 2, 3]
```

Visually:



A triangle (3 verts) produces only `t=0` — the second iteration `t=1` fails `t+2 < 3` and the loop exits.

3.3 UV Clipping

`clip_triangle_near_plane_uv` does the same clipping but carries UV coordinates alongside positions. The same t parameter interpolates both:

$$u_{\text{clip}} = u_{\text{prev}} + t\big(u_{\text{curr}} - u_{\text{prev}}\big)$$

This keeps texture coordinates correct on clipped edges — without this, a clipped triangle would sample the wrong part of the texture.

4 Rasterization

After projection to screen space, the pipeline determines which pixels each triangle covers and what color to write.

4.1 Degenerate Triangle Rejection

Before rasterizing, two checks discard triangles that have no visible area.

Collapsed edge — any edge shorter than `EDGE_THRESHOLD` (0.1 px squared) means two vertices landed on the same pixel:

$$|\boldsymbol{e}_{01}|^2 = \Delta x_{01}^2 + \Delta y_{01}^2 < 0.01 \quad \rightarrow \quad \text{skip}$$

Squared length is used to avoid a `sqrtf` call. All three edges are checked — any single collapsed edge makes the triangle degenerate.

Sub-pixel area — the signed screen-space area must be at least `AREA_THRESHOLD` (0.5 px^2). Catches triangles that are edge-on or otherwise zero-area even without a collapsed edge:

$$|\text{area}| = |(V_2 - V_0) \times (V_1 - V_0)| < 0.5 \quad \rightarrow \quad \text{skip}$$

4.2 Bounding Box

Rather than testing every screen pixel against the triangle, the rasterizer computes the axis-aligned bounding box of the three projected vertices and only loops over that region.

$$x_{\min} = \min\left(V_{0_x}, V_{1_x}, V_{2_x}\right), \quad x_{\max} = \max\left(V_{0_x}, V_{1_x}, V_{2_x}\right)$$

Convert to integer pixels: floor the min (to not miss a left/top edge pixel), ceil the max (to not miss a right/bottom edge pixel), then clamp to screen bounds:

```c
// from compute_bbox() in renderer/geometry.c
*bx0 = (int)fmaxf(0.0f, floorf(mnx));
*bx1 = (int)fminf((float)(screen_width - 1), ceilf(mxx));
```

If after clamping `bx0 > bx1` or `by0 > by1`, the triangle is entirely off-screen — skip.

4.3 Edge Function

The edge function computes the signed area of the parallelogram formed by edge $\boldsymbol{AB}$ and point $\boldsymbol{P}$:

$$E(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{P}) = (P_x - A_x)(B_y - A_y) - (P_y - A_y)(B_x - A_x)$$

- Positive: P is to the left of $\boldsymbol{A} \rightarrow \boldsymbol{B}$ (inside for CCW winding)
- Zero: P is exactly on the edge
- Negative: P is outside

```c
// from edgeFunction() in renderer/geometry.c
static inline float edgeFunction(vec3f a, vec3f b, vec3f c) {
  return (c.x - a.x) * (b.y - a.y) - (c.y - a.y) * (b.x - a.x);
}
```

4.4 Barycentric Coordinates

A point $\boldsymbol{P}$ is inside triangle $(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$ when the edge function is positive for all three edges. The three values, normalised by the total triangle area, are the **barycentric coordinates** (u, v, w):

$$u = \frac{E(\boldsymbol{B}, \boldsymbol{C}, \boldsymbol{P})}{\text{area}}, \quad v = \frac{E(\boldsymbol{C}, \boldsymbol{A}, \boldsymbol{P})}{\text{area}}, \quad w = \frac{E(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{P})}{\text{area}}$$

They always sum to 1: $u + v + w = 1$. Each coordinate is the weight of the opposite vertex. Used to interpolate any per-vertex attribute (UV, depth, normals) across the triangle:

$$\text{attr}_P = u \cdot \text{attr}_A + v \cdot \text{attr}_B + w \cdot \text{attr}_C$$

4.5 Top-Left Rule

When a pixel centre falls exactly on a shared edge between two adjacent triangles, both triangles would claim it — causing a pixel to be drawn twice. The **top-left rule** breaks the tie: a pixel on an edge is only drawn by the triangle for which that edge is a **top edge** (horizontal, going right) or a **left edge** (going down).

```c
// from is_top_left_edge() in renderer/geometry.c
static inline bool is_top_left_edge(vec3f a, vec3f b) {
  return (a.y == b.y && a.x > b.x) || (a.y < b.y);
}
```

Pixels exactly on a non-top-left edge use `> 0` (strict), excluding them. Pixels on a top-left edge use `>= 0`, including them.

4.6 Depth Test

Each pixel in the framebuffer has a corresponding entry in the depth buffer (`depthbuffer`). Before writing a pixel color, the rasterizer checks whether the new fragment is closer than whatever is already there.

Depth is stored as $\frac{1}{z}$ (inverse depth, also called **w-buffer**). $\frac{1}{z}$ interpolates linearly in screen space, whereas raw z does not:

$$\left(\frac{1}{z}\right)_P = u \cdot \left(\frac{1}{z}\right)_A + v \cdot \left(\frac{1}{z}\right)_B + w \cdot \left(\frac{1}{z}\right)_C$$

Larger $\frac{1}{z}$ means smaller z — i.e. closer to the camera. The test is therefore:

```c
float inv_z = u * inv_z_a + v * inv_z_b + w * inv_z_c;
if (inv_z <= renderer->depthbuffer[idx]) continue; // further away - skip
renderer->depthbuffer[idx] = inv_z;
```

4.7 Conservative Floating-Point Edge Bias

On shared edges between adjacent triangles, floating-point rounding can cause one triangle to evaluate an edge function as slightly negative for a pixel that geometrically belongs to it. The result: a one-pixel gap between the triangles.

The fix is to relax the "inside" test by a conservative error bound — `fp_bias`:

```c
float bias = -st->fp_bias;
if (w0 >= bias && w1 >= bias && w2 >= bias)  // pixel passes
```

`fp_bias` is computed from the **magnitude** of the edge function value. For edge $w_0 = E(\boldsymbol{V}_2, \boldsymbol{V}_3, \boldsymbol{P})$:

$$w_0 = \left(P_x - V_{2_x}\right)\left(V_{3_y} - V_{2_y}\right) - \left(P_y - V_{2_y}\right)\left(V_{3_x} - V_{2_x}\right)$$

The floating-point error in this product is proportional to the magnitude of each term. Since P is on screen, $|P_x| \leq \text{screen\_width}$, so:

$$|w_0| \leq \left(|V_{2_x}| + W\right) \cdot |\Delta x_0| + \left(|V_{2_y}| + H\right) \cdot |\Delta y_0|$$

where $\Delta x_0 = V_{3_y} - V_{2_y}$ and $\Delta y_0 = V_{2_x} - V_{3_x}$ are the edge step components, and W/H are screen dimensions.

That bound $\times\ 2 \cdot \varepsilon_{\text{float}}$ gives the actual rounding error. Take the max over all three edges:

```c
// from compute_fp_edge_bias() in renderer/geometry.c
float b0 = (fabsf(proj[1].x) + sw) * fabsf(dx_w0)
         + (fabsf(proj[1].y) + sh) * fabsf(dy_w0);
// ... b1, b2 for other edges ...
return fmaxf(fmaxf(b0, b1), b2) * 2.0f * FLT_EPSILON + seam_bias;
```

`seam_bias` is a user-tunable extra tolerance (`settings.seam_bias`) for cases where the computed bound is still too tight.

5 Color Math

5.1 HSV to RGB

HSV separates color into three intuitive components:

- **H** (hue) — angle on the color wheel, $[0°, 360°)$
- **S** (saturation) — color purity, $[0, 1]$. 0 = grey, 1 = full color
- **V** (value) — brightness, $[0, 1]$

Used in the renderer to assign each triangle a distinct flat color from its index:

```c
float hue = fmodf((float)triangle_index * 10.0f, 360.0f);
Color flat_color = hsv_to_rgb(hue, 1.0f, 1.0f);
```

5.1.1 Chroma and secondary component

Chroma c is the color intensity — how far the color is from grey:

$$c = V \times S$$

At full saturation $c = V$. At zero saturation $c = 0$ (grey regardless of hue).

The **secondary component** x is a triangle wave that ramps $0 \to c \to 0$ within each 60° sector:

$$x = c \times \left(1 - \left|\left(\frac{H}{60}\right) \bmod 2 - 1\right|\right)$$

`fmodf(H/60, 2)` counts 0→2 across each pair of sectors. Subtracting 1 and taking the absolute value folds that into a 0→1→0 triangle. Multiplying by c scales it.

5.1.2 Sector mapping

The hue circle is divided into six 60° sectors, each corresponding to a transition between two primary colors:

hue range	transition	R	G	B
0° – 60°	red → yellow	c	x	0
60° – 120°	yellow → green	x	c	0
120° – 180°	green → cyan	0	c	x
180° – 240°	cyan → blue	0	x	c
240° – 300°	blue → magenta	x	0	c
300° – 360°	magenta → red	c	0	x

5.1.3 Match value

After the sector lookup, all three channels are shifted up by $m = V - c$ so the result reaches the correct brightness:

$$(R, G, B) = (r + m,\quad g + m,\quad b + m) \times 255$$

At $V = 1, S = 1$: $m = 0$, full saturation, output is in $[0, 255]$. At $V = 1, S = 0$: $c = 0$, $x = 0$, $m = 1$ — all channels $= 255 =$ white.

```c
// from hsv_to_rgb() in math/color.c
float c = v * s;
float x = c * (1 - fabsf(fmodf(h / 60.0f, 2) - 1));
float m = v - c;
// ... sector table ...
Color color = {
  (unsigned char)((r + m) * 255),
  (unsigned char)((g + m) * 255),
  (unsigned char)((b + m) * 255), 255
};
```

6 Lighting

6.1 Flat (Lambertian) Shading

One shade per triangle. Reuses the face normal already computed for backface culling (see Section 2.3.1) — no extra per-vertex data required.

Lambert's cosine law: diffuse reflectance is proportional to the cosine of the angle between the surface normal and the direction to the light.

$$I = k_a + k_d \cdot \max\left(0, \hat{n} \cdot \hat{l}\right)$$

- $\hat{n}$ — face normal, camera space (same one used for the backface test)
- $\hat{l}$ — direction **toward** the light, camera space
- k_a — ambient term: a brightness floor so unlit faces aren't pure black
- k_d — diffuse coefficient, typically $1 - k_a$

The $\max(0, \cdot)$ clamp matters: a negative dot product means the light is behind the face relative to its normal, which should contribute zero light, not negative light.

Where the result lives: I is stored as a `vec3f light_rgb` on `screen_tri_t` — an RGB triple, not a single scalar, even though stage 1 has only one white light (I, I, I). This is deliberate: stage 2 sums multiple **colored** lights into this same field, so the data layout doesn't need to change again when lights gain color.

Applying to color — at the pixel, not at packaging time: `screen_tri_t` can resolve to either `flat_color` or a texture sample, decided per-pixel in `fetch_pixel_color()`. Scaling `flat_color` alone (as an earlier draft of this section did) would silently skip lighting on every textured triangle. The scale has to happen after that choice is made, at the single return point:

$$(R', G', B') = (R, G, B) \odot I$$

($\odot$ = component-wise product — each channel scaled by its own light term.)

```c
// build_screen_tris() in renderer/geometry.c - normal is a second call to
// compute_face_normal(), the same cross product is_backfacing() computes
// internally and discards
float ndotl = fmaxf(0.0f, vec_dot(normal, light_dir_cam));
float intensity = ambient + (1.0f - ambient) * ndotl;
vec3f light_rgb = {intensity, intensity, intensity}; // white light; stage 2
                                                     // sums per-light color here
```

```c
// fetch_pixel_color() in renderer/draw.c - single exit point, covers both
// the flat_color and texture-sample branches
Color base = st->tex ? /* perspective-correct sample */ : st->flat_color;
return color_scale(base, st->light_rgb);
```

Why camera space: by the point `build_screen_tris` runs, vertices (and the derived normal) are already in camera space. `light_dir` must be rotated into the same space once per frame, the same way vertices are — but as a **direction**, so only rotation applies, no translation:

$$\hat{l}_{\text{cam}} = \text{rotate}_x\Big(\text{rotate}_y\big(\hat{l}_{\text{world}}, -\text{yaw}\big), -\text{pitch}\Big)$$

Limitation: one intensity per face means flat-shaded facets are visible on curved meshes (e.g. Suzanne) — each triangle is a uniform patch, with hard edges between them. Smooth shading (Gouraud/Phong) fixes this but requires per-vertex normals, which the current non-indexed `mesh_s` layout does not store.

7 Tile-Based Rendering

The screen is divided into fixed-size tiles (`TILE_SIZE` × `TILE_SIZE` pixels). Instead of rendering triangles one at a time across the whole screen, the pipeline first bins each triangle into the tiles it overlaps, then renders each tile independently. This makes phase 3 trivially parallelizable — tiles share no state.

7.1 Phase 2a — Counting Pass

Zero all tile counters, then walk every screen triangle. For each triangle, convert its pixel bbox to tile coordinates:

$$tx_0 = \left\lfloor \frac{x_0}{\text{TILE\_SIZE}} \right\rfloor, \quad tx_1 = \left\lfloor \frac{x_1}{\text{TILE\_SIZE}} \right\rfloor$$

Integer divide rounds down — correct since you want the tile **containing** the pixel. Clamp to `[0, tiles_x - 1]` in case rounding pushes past the last tile.

Then increment every tile in the range:

```c
for (int ty = ty0; ty <= ty1; ty++)
    for (int tx = tx0; tx <= tx1; tx++)
        tile_count_buf[ty * tiles_x + tx]++;
```

`ty * tiles_x + tx` is the flat tile index (row-major, matches screen layout). A triangle spanning 3×3 tiles increments 9 counters; a small triangle in one tile increments 1.

After this pass: `tile_count_buf[ti]` = number of triangles that overlap tile `ti`.

7.2 Phase 2b — Prefix Sum

Convert counts into start offsets so each tile knows where its triangle list begins in the flat `bin_buf[]` array:

$$\text{start}[0] = 0, \quad \text{start}[i] = \text{start}[i-1] + \text{count}[i-1]$$

```c
int total_bin = 0;
for (int ti = 0; ti < num_tiles; ti++) {
    tile_start_buf[ti] = total_bin;
    total_bin += tile_count_buf[ti];
}
```

Example with 4 tiles:

tile	count	start
0	3	0
1	1	3
2	5	4
3	2	9

After this: `tile_start_buf[ti]` points to the first free slot for tile `ti` in `bin_buf[]`. Total entries needed = `total_bin`.

7.3 Phase 2c — Fill Pass

Reset `tile_count_buf` to 0 — reuse it as a per-tile write cursor. Walk triangles again with the same bbox loop. For each overlapping tile:

```c
int idx = tile_start_buf[ti] + tile_count_buf[ti];
bin_buf[idx] = si;       // write triangle index
tile_count_buf[ti]++;    // advance write cursor
```

`tile_start_buf[ti]` is the base for this tile; `tile_count_buf[ti]` counts how many have been written so far. Together they always point to the next free slot.

After this pass: `bin_buf[]` is a flat array of triangle indices. For tile `ti`, the relevant indices are:

```
bin_buf[ tile_start_buf[ti] .. tile_start_buf[ti] + tile_count_buf[ti] ]
```

No pointers, no dynamic allocation — cache-friendly, safe to read in parallel.

8 Focal Length and Field of View

`focal_length` controls how wide the camera sees. Conceptually it is the distance (in NDC units) from the camera to the projection plane.

A vertex at $(x, 0, z)$ projects to $x_{\text{proj}} = (x/z) \cdot f$.

- Large f: tight "telephoto" lens. Objects appear larger; narrow field of view.
- Small f: wide "fisheye" lens. Objects appear smaller; wide field of view.

The relationship to field of view angle θ (horizontal half-angle):

$$f = \frac{1}{\tan\left(\frac{\theta}{2}\right)}$$

At $\theta = 90°$: $f = 1$. At $\theta = 60°$: $f \approx 1.73$.